

HW5

HW4 的 30 個 DE genes

```
1 # HW4 的 30 個 DE genes
2 MLL_train <- read.delim("MLL_train.txt")
3 data <- MLL_train[-c(1:49), ]
4
5 # 正規化
6 min_max_norm <- function(x){
7   (x - min(x)) / (max(x) - min(x))
8 }
9
10 # 對每一 row 計算 t-test
11 count_t <- function(group_a, group_b){
12   t_value <- numeric(nrow(group_a))
13   for(i in 1:nrow(group_a)){
14     t_value[i] <- t.test(group_a[i, ], group_b[i, ])$statistic
15   }
16   return(t_value)
17 }
18
19 # 2. Normalize data by each row(0~1)
20 data_norm <- t(apply(data[, 3:ncol(data)], 1, min_max_norm))
21
22 col_name <- colnames(data_norm)
23 labels <- sub("-", ".", col_name)
24 table(labels)
25
26 # ALL
27 ALL_data <- data_norm[, which(labels == "ALL")] # 只有 ALL 的欄位資料
28 ALL_others_data <- data_norm[, -which(labels == "ALL")] # 除了 ALL 的欄位資料
29
30 ALL_mean <- apply(ALL_data, 1, mean) # 計算只有 ALL 欄位的每一列平均值
31 ALL_others_data_mean <- apply(ALL_others_data, 1, mean) # 計算除了 ALL 欄位的每一列平均值
32
33 # 只保留 mean(該組) > mean(其他)
34 ALL_data_filter <- ALL_data[which(ALL_mean > ALL_others_data_mean), ]
35 ALL_others_data_filter <- ALL_others_data[which(ALL_mean > ALL_others_data_mean), ]
36
37 # 計算 t-test
38 ALL_t <- count_t(ALL_data_filter, ALL_others_data_filter)
39
40 # 由大到小排列
41 ALL_t_sort <- order(ALL_t, decreasing = TRUE)
42
43 # 透過 order 回傳的前十個索引找到在 ALL_data_filter 的 gene
44 ALL_top_10_genes <- ALL_data_filter[ALL_t_sort[1:10], ]
45 ALL_top_10_genes
46
47 ## 以下同理
48
49 # MLL
50 MLL_data <- data_norm[, which(labels == "MLL")]
51 MLL_others_data <- data_norm[, -which(labels == "MLL")]
52
53 MLL_mean <- apply(MLL_data, 1, mean) # 計算只有 MLL 欄位的每一列平均值
54 MLL_others_data_mean <- apply(MLL_others_data, 1, mean) # 計算除了 MLL 欄位的每一列平均值
55
56 # 只保留 mean(該組) > mean(其他)
57 MLL_data_filter <- MLL_data[which(MLL_mean > MLL_others_data_mean), ]
58 MLL_others_data_filter <- MLL_others_data[which(MLL_mean > MLL_others_data_mean), ]
59
60 MLL_t <- count_t(MLL_data_filter, MLL_others_data_filter)
61 MLL_t_sort <- order(MLL_t, decreasing = TRUE)
62
63 MLL_top_10_genes <- MLL_data_filter[MLL_t_sort[1:10], ]
64 MLL_top_10_genes
65
```


PCA

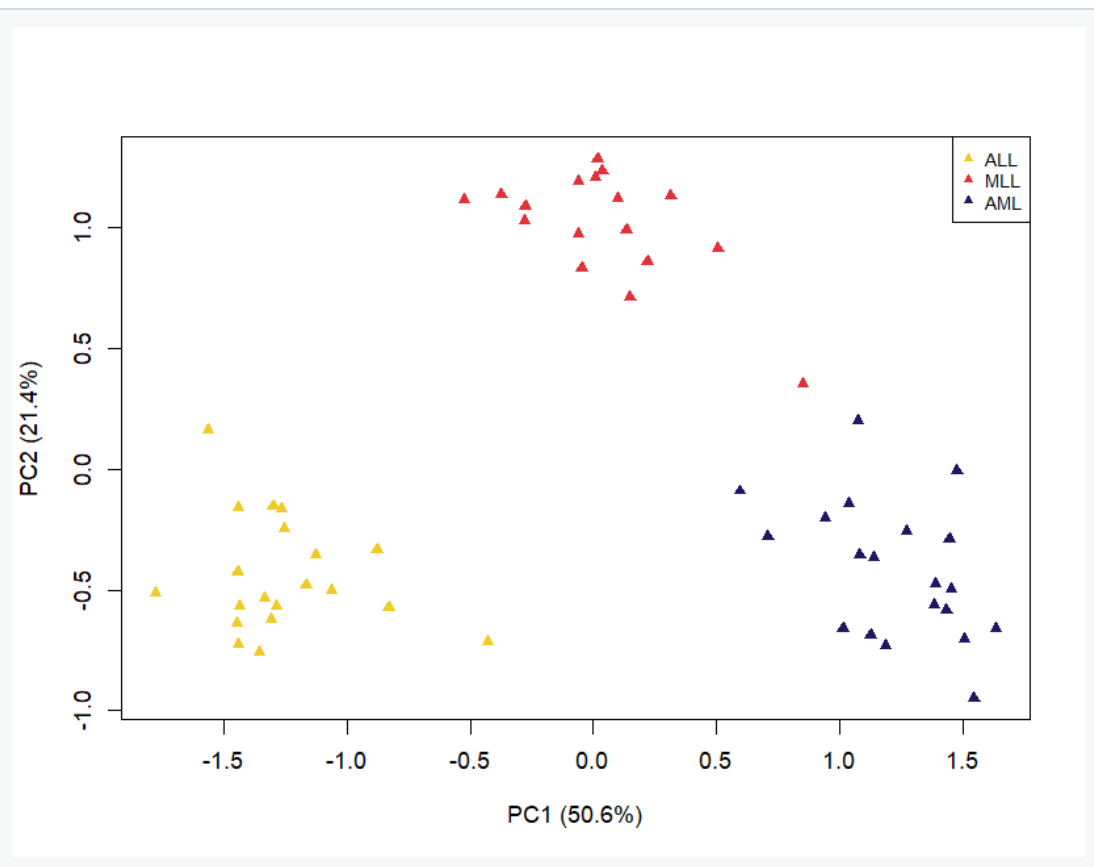
```
my_data_prcomp <- prcomp(t(my_data))

# 畫出 PC1、PC2 個別主導幾 %
pv <- summary(my_data_prcomp)$importance[2, 1:2] * 100
xlab_txt <- paste0("PC1 (", round(pv[1], 1), "%)")
ylab_txt <- paste0("PC2 (", round(pv[2], 1), "%)")

plot(my_data_prcomp$x[, 1],
     my_data_prcomp$x[, 2], col = col_list, pch = 17,
     xlab = xlab_txt, ylab = ylab_txt)

legend("topright", legend = names(color),
     col = color[names(color)], pch = 17, cex = 0.8)
```

Output:



MDS

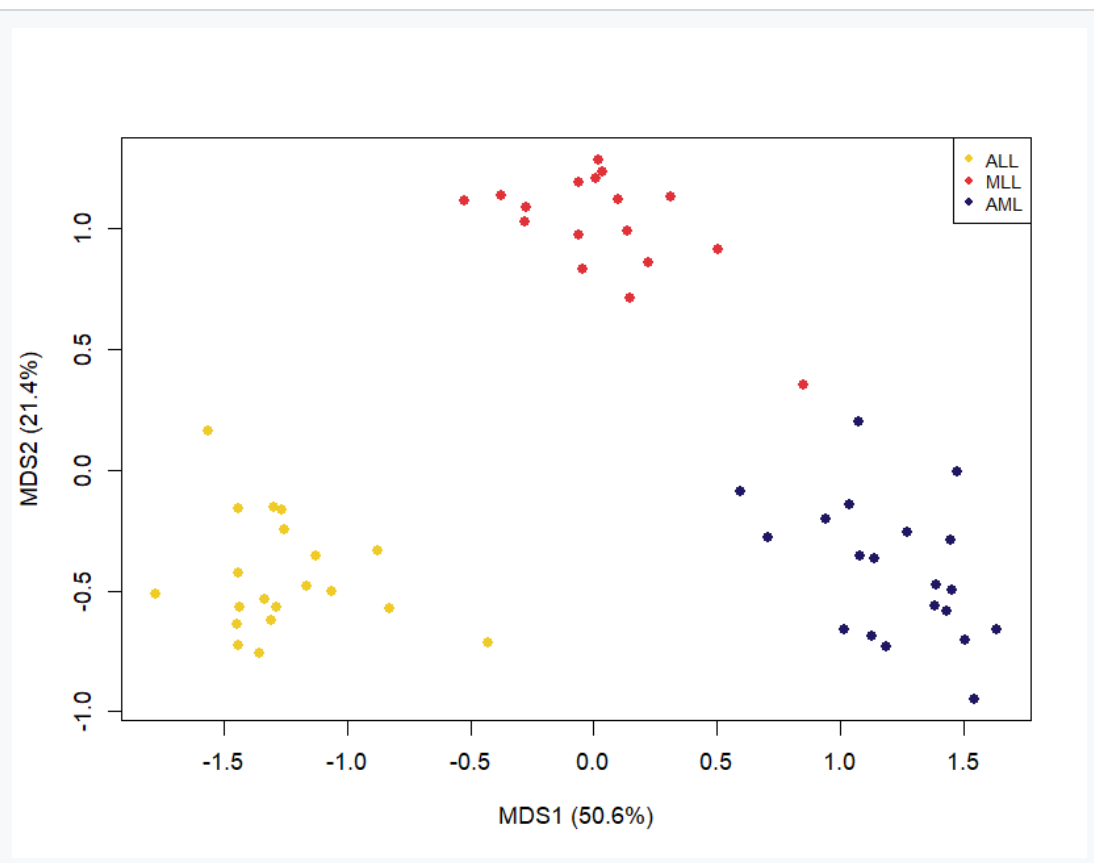
```
my_data_mds <- cmdscale(dist(t(my_data)), k = 2, eig = TRUE)
# 特徵值 → 每個維度解釋的比例
eig <- my_data_mds$eig
## 只用正特徵值當分母
pv_mds <- eig[1:2] / sum(eig[eig > 0]) * 100

xlab_mds <- paste0("MDS1 (", round(pv_mds[1], 1), "%)")
ylab_mds <- paste0("MDS2 (", round(pv_mds[2], 1), "%)")

plot(my_data_mds$points[, 1], my_data_mds$points[, 2],
     col = col_list, pch = 16,
     xlab = xlab_mds, ylab = ylab_mds)

legend("topright", legend = names(color), col = color,
     pch = 16, cex = 0.8)
```

Output:



AI

PCS有summary 那MDS有嗎

探索MDS特征值提取以获取方差解释比例。

有,MDS 也能算出每個維度佔的變異百分比,只是不是用 `summary()`,而是要在 `cmdscale` 裡加 `eig = TRUE` 拿特徵值(eigenvalues)自己算。

你現在寫的是:

```
r
my_data_mds <- cmdscale(dist(t(my_data)), 2)
```

這樣只回傳座標,沒有特徵值。改成這樣就拿到:

```
r
my_data_mds <- cmdscale(dist(t(my_data)), k = 2, eig = TRUE)

# 特徵值 → 每個維度解釋的比例
eig <- my_data_mds$eig
pv_mds <- eig[1:2] / sum(eig[eig > 0]) * 100 # 只用正特徵值當分母

xlab_mds <- paste0("MDS1 (", round(pv_mds[1], 1), "%)")
ylab_mds <- paste0("MDS2 (", round(pv_mds[2], 1), "%)")

plot(my_data_mds$points[, 1], my_data_mds$points[, 2],
     col = col_list, pch = 16, xlab = xlab_mds, ylab = ylab_mds)

legend("topright", legend = names(color), col = color, pch = 16, cex = 0.8)
```

幾個要注意的點:

座標的取法變了。加了 `eig = TRUE` 之後, `cmdscale` 回傳的是一個 list,座標要用 `$points` 取 (原本沒加 `eig` 時直接就是矩陣)。所以畫圖那行要從 `my_data_mds[, 1]` 改成 `my_data_mds$points[, 1]`,不然會報錯。

分母為什麼用 `eig[eig > 0]`。classical MDS(PCoA)在非歐氏距離下可能出現負的特徵值。你這裡用的是 Euclidean distance,理論上不會有負值,直接 `sum(eig)` 也可以;但用 `sum(eig[eig > 0])` 比較保險,是通用寫法。

2. 用同樣的 30 個 DE genes,畫一張 heatmap + dendrogram

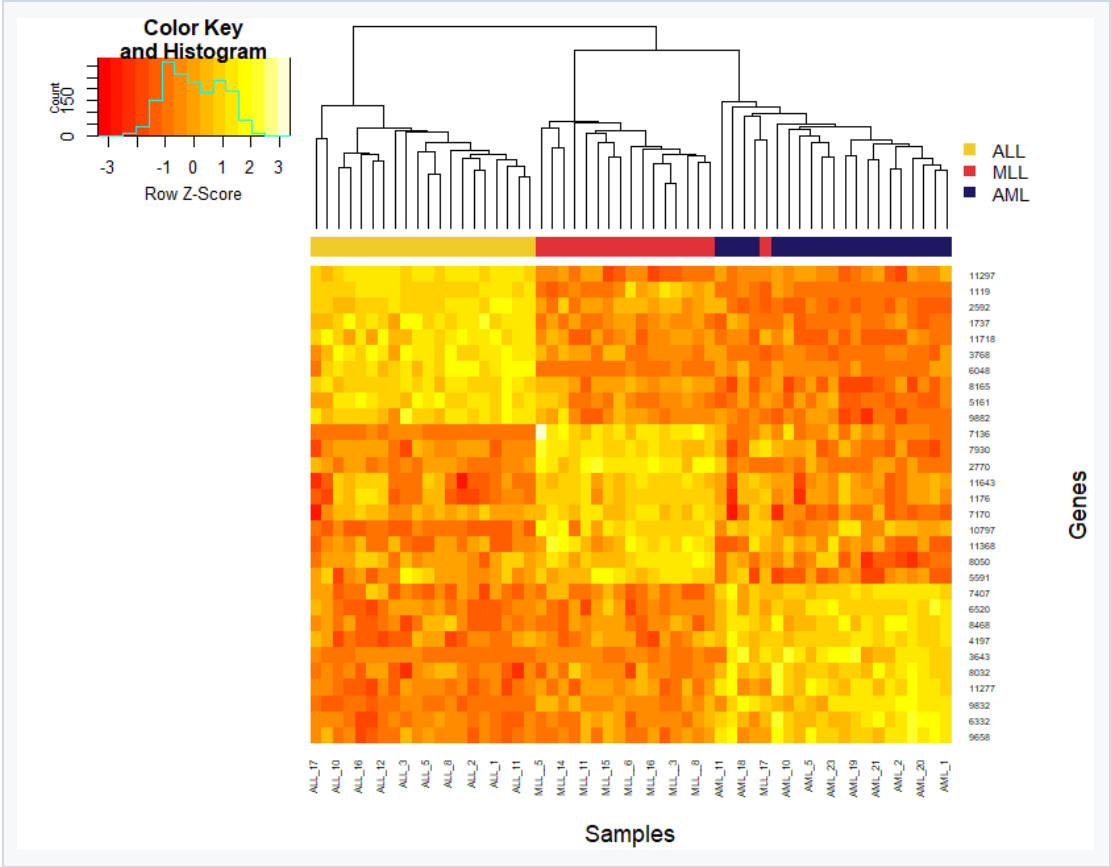
• 條件 : Average linkage + Euclidean distance

```
# 計算距離的方法 : euclidean
my_dist <- function(x) {
  dist(x, method = "euclidean")
}
# 分群方法 : average
my_hclust <- function(d) {
  hclust(d, method = "average")
}

heatmap.2(my_data,
  Rowv = F, Colv = TRUE,
  dendrogram = "both",
  distfun = my_dist,
  hclustfun = my_hclust,
  scale = "row",
  trace = "none",
  # 樣本上方加一條 ALL/MLL/AML 顏色條
  ColSideColors = col_list,
  # (欄邊界, 列邊界), 避免標籤被切
  margins = c(6, 8),
  # 30 個基因名縮小才放得下
  cexRow = 0.6,
  cexCol = 0.7,
  xlab = "Samples", ylab = "Genes")

legend("topright", legend = names(color), fill = color,
  border = NA, bty = "n", cex = 0.8)
```

Output:



AI

```
# HW4 的 30 個 DE genes
MLL_train <- read.delim(
  "MLL_train.txt") data <-
  MLL_train[,c(1:49)] # 正
  用 fit_min_max_norm <- fu
  nction(x){ (x - min(x)) / ...
```

PASTED

這是我完整的程式碼 幫我針對看熱圖的部分怎麼修改

追蹤程式碼找出變數名稱不匹配問題並優化熱圖。

找到問題了。你這張圖之所以只有 4 列,最關鍵的原因是變數名稱打錯——這是你熱圖那段唯一真正的 bug。

核心錯誤: `my.data` vs `my_data`

你前面辛苦算出來、有 30 列的物件叫做 `my_data` (底線),但 `heatmap.2()` 裡傳進去的卻是 `my.data` (點):

```
r
my_data <- data_norm[selected_genes, ] # ← 你真正要的 30 個基因
...
heatmap.2(my.data, ...) # ← 卻畫了 my.data
```

在 R 裡 `.` 和 `_` 是完全不同的字元,`my.data` 跟 `my_data` 是兩個不同的物件。你的環境裡大概還殘留著某個舊的、只有 4 列的 `my.data` (可能是之前測試留下來的),所以 `heatmap.2` 就拿那個舊物件去畫,才會出現 4 條全叫 "Gene" 的列。把它改成 `my_data`,30 個基因就會回來了。

其他要一起補的地方

第一,`heatmap.2` 來自 `gplots` 套件,記得先載入,否則會找不到函數:

```
r
library(gplots)
```

第二,30 個基因的列標籤要調一下,不然會擠在一起或被切掉。用 `margins` 加大邊界、`cexRow` 縮小字體。

小字體。

第三(選配但很推薦),既然你有 ALL / MLL / AML 三類,可以用 `ColSideColors` 在樣本上方加一條彩色條,一眼就能對照分群結果跟真實類別是否吻合——這其實是這份作業最有價值的呈現。

改好的完整版

```
r

library(gplots)

my_dist <- function(x) dist(x, method = "euclidean") # Euclidean distance
my_hclust <- function(d) hclust(d, method = "average") # Average linkage

heatmap.2(my_data,                                     # ← 改成 my_data(底線)
          Rowv = TRUE, Colv = TRUE,
          dendrogram = "both",
          distfun = my_dist,
          hclustfun = my_hclust,
          scale = "row",
          trace = "none",
          ColSideColors = col_list,                    # 樣本上方加一條 ALL/MLL/AML 顏色條
          margins = c(6, 8),                          # (欄邊界, 列邊界), 避免標籤被切
          cexRow = 0.6,                                # 30 個基因名縮小才放得下
          cexCol = 0.7,
          xlab = "Samples", ylab = "Genes")

# 幫 ColSideColors 加圖例
legend("topright", legend = names(color), fill = color,
      border = NA, bty = "n", cex = 0.8)
```

一個觀念上的小提醒

你的 `data_norm` 已經對每一列做過 min-max 正規化(0~1),這裡再加 `scale = "row"` 等於又做了一次 z-score 標準化——所以顏色軸才會顯示 "Row Z-Score"。這在視覺上沒問題、也是熱圖常見做法,只是心裡要知道是「兩層轉換」。如果你想讓顏色直接反映原本的 0~1 正規化值,就把 `scale = "row"` 改成 `scale = "none"`,那時 color key 會變成 0 到 1。兩種都合理,看你想強調「基因內的相對高低」(用 row)還是「正規化後的絕對值」(用 none)。

改完後先跑一次 `dim(my_data)` 確認是 30 x 樣本數,再畫圖,就應該會看到 30 列、每類基因在對應樣本群裡呈現漂亮的區塊了。